BÁO CÁO BÀI TẬP VỀ NHÀ NGÀY 26/05/2026

HỌ VÀ TÊN: PHAN MINH VƯỢNG

MSSV: 20225241

1. Bài 1 – HTTP_operator

- Mã nguồn

```c
#include <arpa/inet.h>
#include <signal.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

void signalHandler(int signo) { // Xử lý sự kiện tiến trình con kết thúc
    printf("signo = %d\n", signo);

    while (waitpid(-1, NULL, WNOHANG) > 0) {
        printf("A child process terminated.\n");
    }

    return;
}

// ham gui response html ve trinh duyet
void send_response(int client, char *html) {
    char response[8192];

    sprintf(response,
```

```
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/html; charset=UTF-8\r\n"
            "Content-Length: %ld\r\n"
            "Connection: close\r\n"
            "\r\n"
            "%s",
            strlen(html), html);

    send(client, response, strlen(response), 0);
}

// ham gui trang loi ve trinh duyet
void send_error_page(int client, char *message) {
    char html[4096];

    sprintf(html,
            "<!DOCTYPE html>"
            "<html>"
            "<head>"
            "<meta charset='UTF-8'>"
            "<title>Lỗi</title>"
            "</head>"
            "<body>"
            "<h2>Lỗi</h2>"
            "<p>%s</p>"
            "</body>"
            "</html>",
            message);

    send_response(client, html);
}
```

```c
// ham lay gia tri cua tham so trong query string hoac body
void get_param(char *data, char *key, char *value) {
    value[0] = '\0';

    char temp[1024];
    strcpy(temp, data);

    char *token = strtok(temp, "&");

    while (token != NULL) {
        char param_key[64];
        char param_value[64];

        int n = sscanf(token, "%[^=]=%s", param_key, param_value);

        if (n == 2 && strcmp(param_key, key) == 0) {
            strcpy(value, param_value);
            return;
        }

        token = strtok(NULL, "&");
    }
}

// ham xu ly phep tinh va gui ket qua ve trinh duyet
void handle_calculate(int client, char *data) {
    char a_str[64];
    char b_str[64];
    char op[64];

    get_param(data, "a", a_str);
    get_param(data, "b", b_str);
```

```
    get_param(data, "op", op);

    // kiem tra du tham so chua
    if (strlen(a_str) == 0 || strlen(b_str) == 0 || strlen(op) == 0) {
        send_error_page(client, "Thiếu tham số. Ví dụ: "
                                "/?a=10&b=5&op=add");
        return;
    }

    double a = atof(a_str);
    double b = atof(b_str);
    double result = 0;

    char op_symbol[8];

    // kiem tra toan tu va tinh ket qua
    if (strcmp(op, "add") == 0) {
        result = a + b;
        strcpy(op_symbol, "+");
    } else if (strcmp(op, "sub") == 0) {
        result = a - b;
        strcpy(op_symbol, "-");
    } else if (strcmp(op, "mul") == 0) {
        result = a * b;
        strcpy(op_symbol, "*");
    } else if (strcmp(op, "div") == 0) {
        if (b == 0) {
            send_error_page(client, "Không thể chia cho 0");
            return;
        }

        result = a / b;
```

```
        strcpy(op_symbol, "/");
    } else {
        send_error_page(client,
                        "Toán tử không hợp lệ. Toán tử hợp lệ: add, sub, "
                        "mul, div.");
        return;
    }

    char html[8192];

    // tao trang html hien thi ket qua
    sprintf(html,
            "<!DOCTYPE html>"
            "<html>"
            "<head>"
            "<meta charset='UTF-8'>"
            "<title>Kết quả phép tính</title>"
            "</head>"
            "<body>"
            "<h2>Kết quả phép tính</h2>"
            "<p>Phép tính: %.2lf %s %.2lf</p>"
            "<h3>Kết quả: %.2lf</h3>"
            "</body>"
            "</html>",
            a, op_symbol, b, result);

    send_response(client, html);
}

int main() {
    int port = 8888;
```

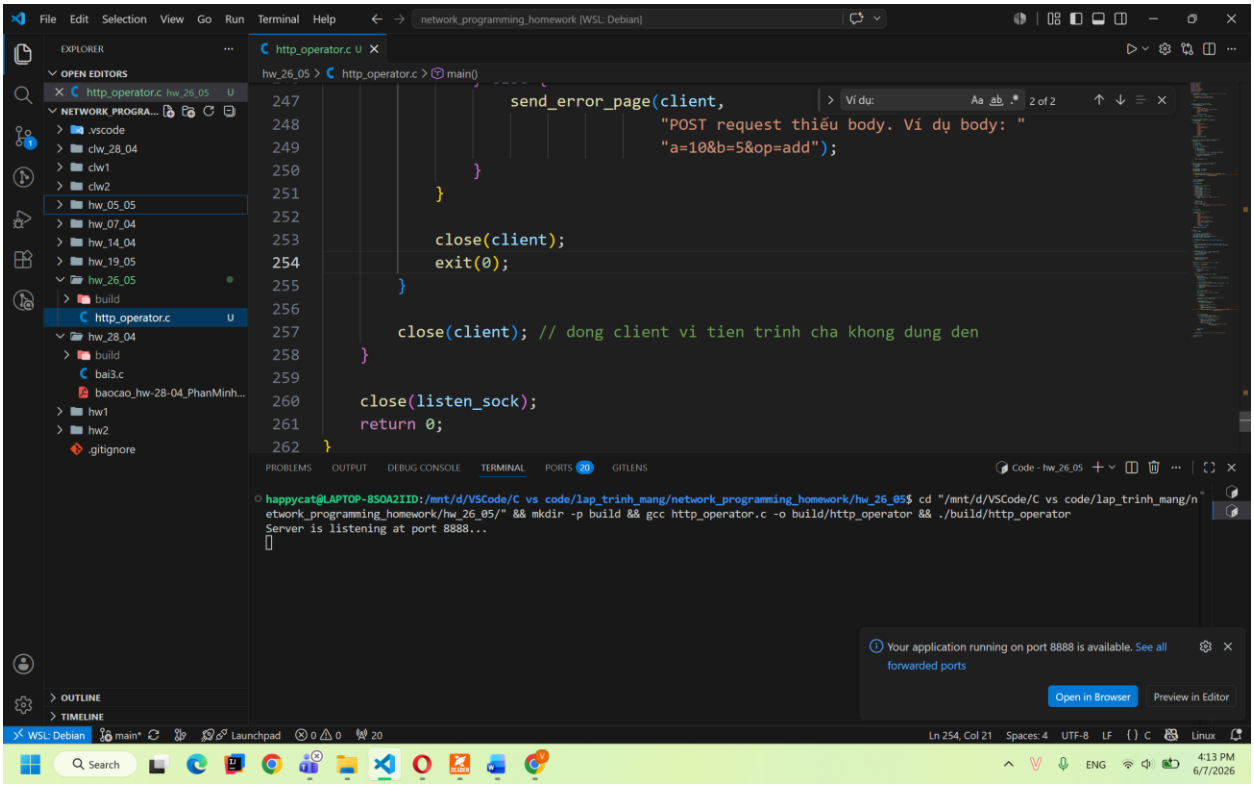
```c
// khai bao dia chi tai cong 8888
struct sockaddr_in server_addr = {0};
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
server_addr.sin_port = htons(port);

// tao listen sock
int listen_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

int opt = 1;

// Thiết lập SO_REUSEADDR để giải phóng cổng ngay khi server tắt
if (setsockopt(listen_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) <
    0) {
    perror("setsockopt failed");
    exit(EXIT_FAILURE);
}

// gan dia chi cho socket
if (bind(listen_sock, (struct sockaddr *)&server_addr,
         sizeof(server_addr)) < 0) {
    perror("bind() failed");
    exit(EXIT_FAILURE);
}

if (listen(listen_sock, 5) < 0) {
    perror("listen() failed");
    exit(EXIT_FAILURE);
}

printf("Server is listening at port %d...\n", port);
```

```c
signal(SIGCHLD, signalHandler);

while (1) {
    int client = accept(listen_sock, NULL, NULL);

    if (client < 0) {
        perror("accept() failed");
        continue;
    }

    printf("New client accepted: %d\n", client);

    if (fork() == 0) {
        close(listen_sock);       // dong listen_sock vi khong dung
        signal(SIGCHLD, SIG_DFL); // tien trinh con khong quan tam SIGCHLD

        char buffer[4096];

        // nhan HTTP request tu client
        int bytes_received = recv(client, buffer, sizeof(buffer) - 1, 0);

        if (bytes_received <= 0) {
            printf("Client number %d disconnected\n", client);
            close(client);
            exit(0);
        }

        buffer[bytes_received] = '\0';

        printf("Request from client:\n%s\n", buffer);

        char method[16];
```

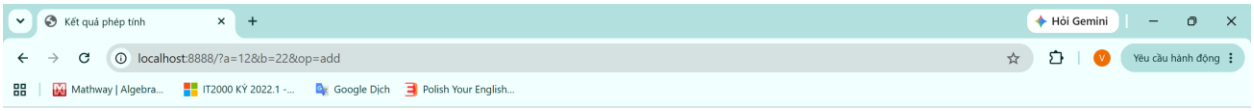
```
        char path[1024];

        // lay method va path tu dong dau tien cua HTTP request
        sscanf(buffer, "%s %s", method, path);

        // neu request la GET
        if (strcmp(method, "GET") == 0) {
            char *query = strchr(path, '?');

            // GET phai co tham so nam sau dau ?
            if (query != NULL) {
                query++; // bo qua ky tu '?'
                handle_calculate(client, query);
            } else {
                send_error_page(client, "GET request thiếu tham số. Ví dụ: "
                                        "/?a=10&b=5&op=add");
            }
        }

        // neu request la POST
        else if (strcmp(method, "POST") == 0) {
            char *body = strstr(buffer, "\r\n\r\n");

            if (body != NULL && strlen(body + 4) > 0) {
                body += 4; // bo qua "\r\n\r\n"
                handle_calculate(client, body);
            } else {
                send_error_page(client,
                                "POST request thiếu body. Ví dụ body: "
                                "a=10&b=5&op=add");
            }
        }
```

```
            close(client);
            exit(0);
        }

        close(client); // dong client vi tien trinh cha khong dung den
    }

    close(listen_sock);
    return 0;
}
```

- Kết quả
    - Chạy server ở cổng 8888



    - Tạo get request bằng trình duyệt (GET /?a=12&b=22&op=add)

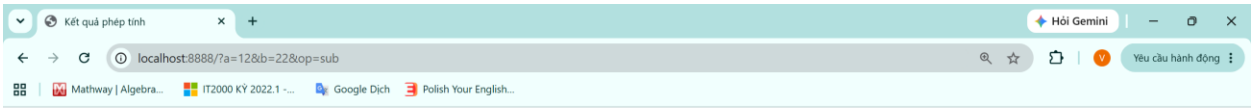


Kết quả phép tính

Phép tính: 12.00 + 22.00

Kết quả: 34.00

- Tạo get request bằng trình duyệt (GET /?a=12&b=22&op=sub)

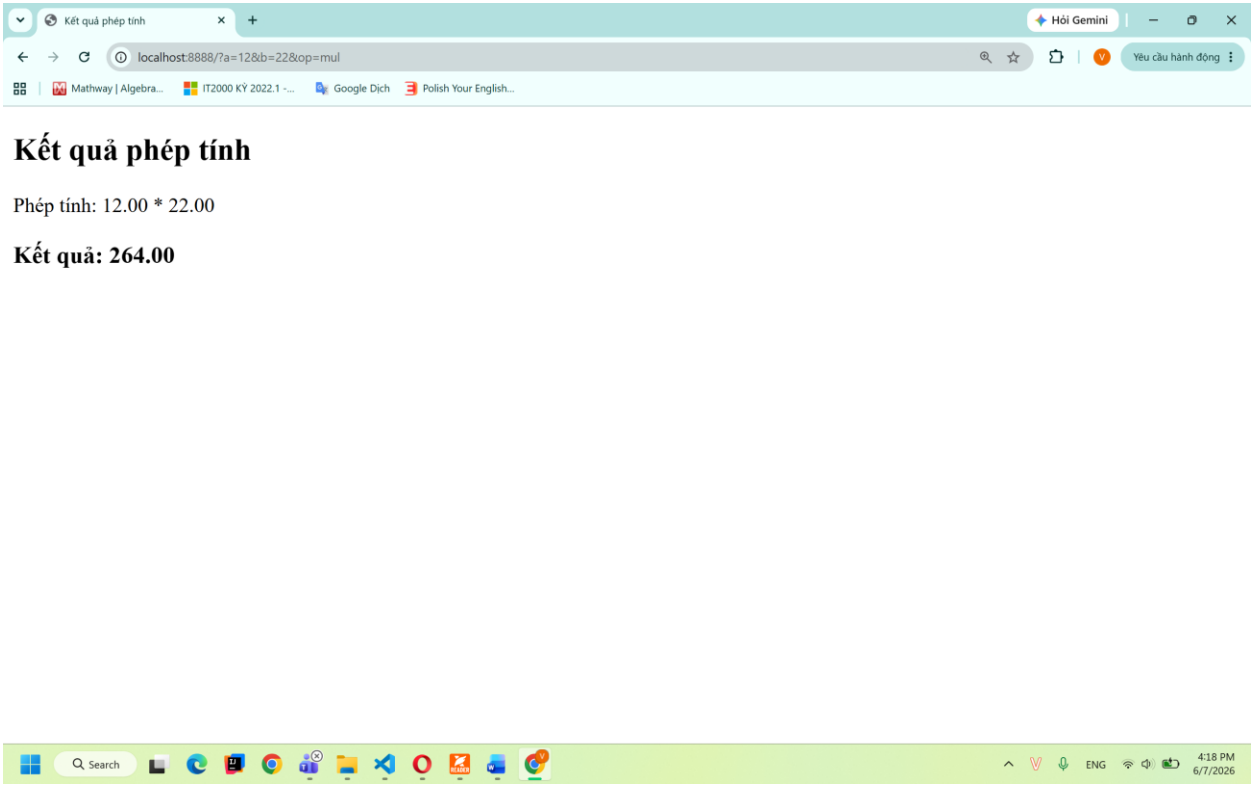


Kết quả phép tính

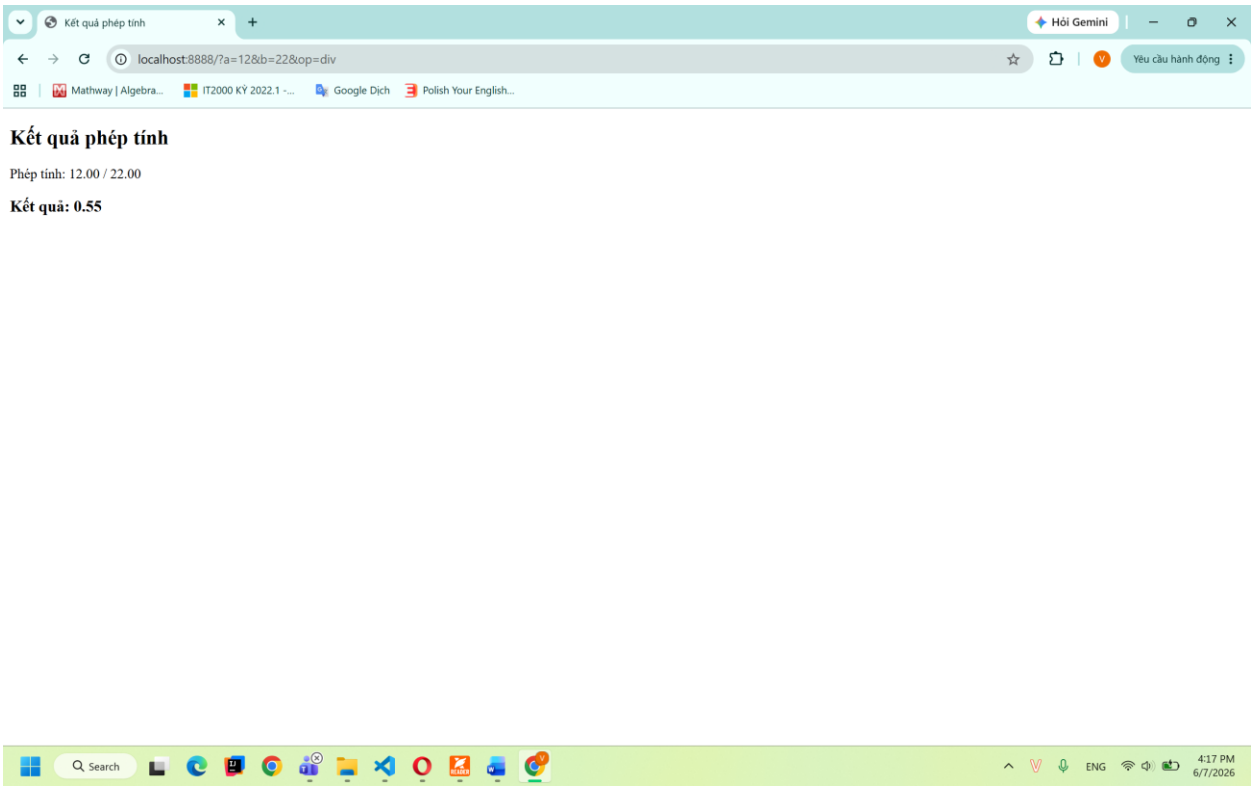
Phép tính: 12.00 - 22.00

Kết quả: -10.00

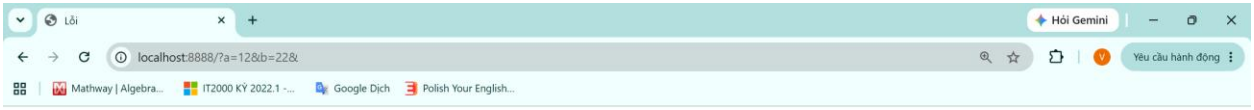
- Tạo get request bằng trình duyệt (GET /?a=12&b=22&op=mul)



- Tạo get request bằng trình duyệt (GET /?a=12&b=22&op=div)

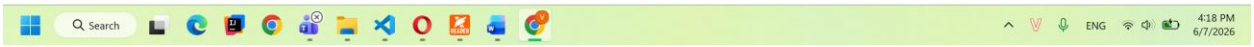


- Tạo get request bằng trình duyệt (trường hợp lỗi)

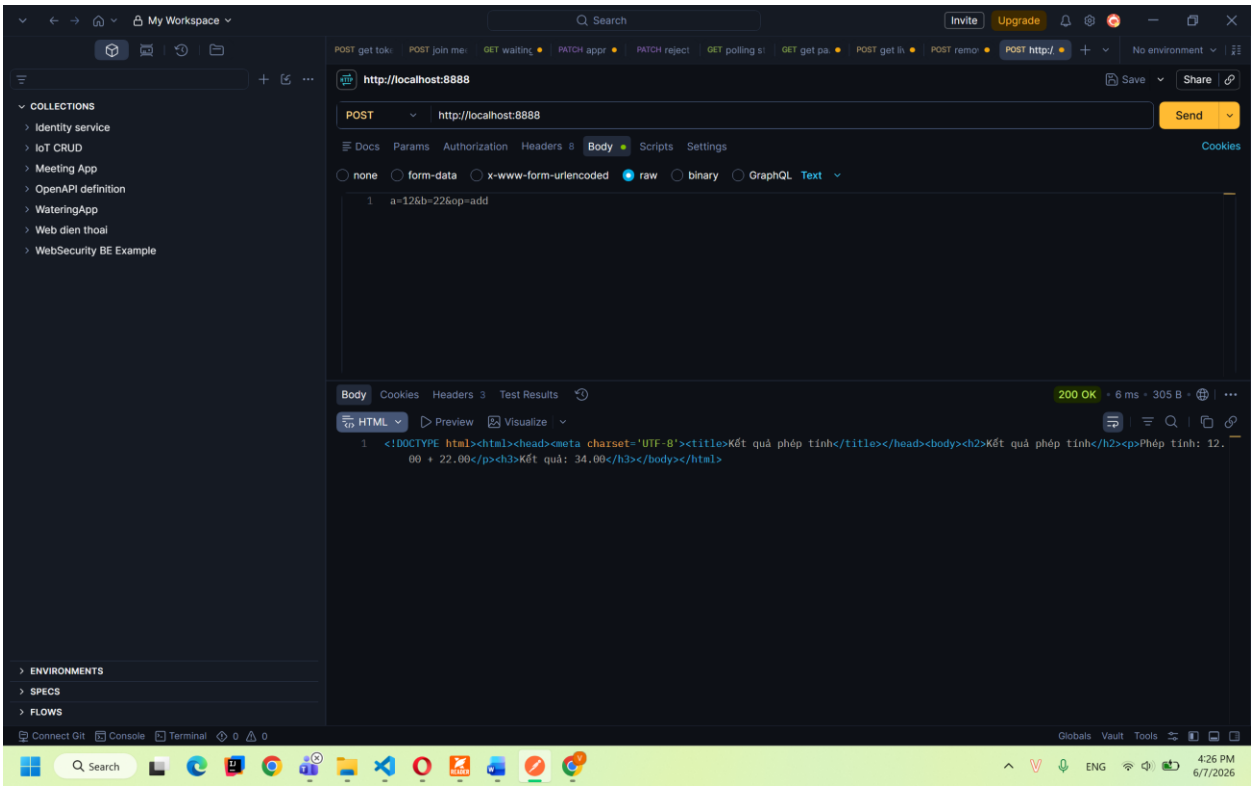


Lỗi

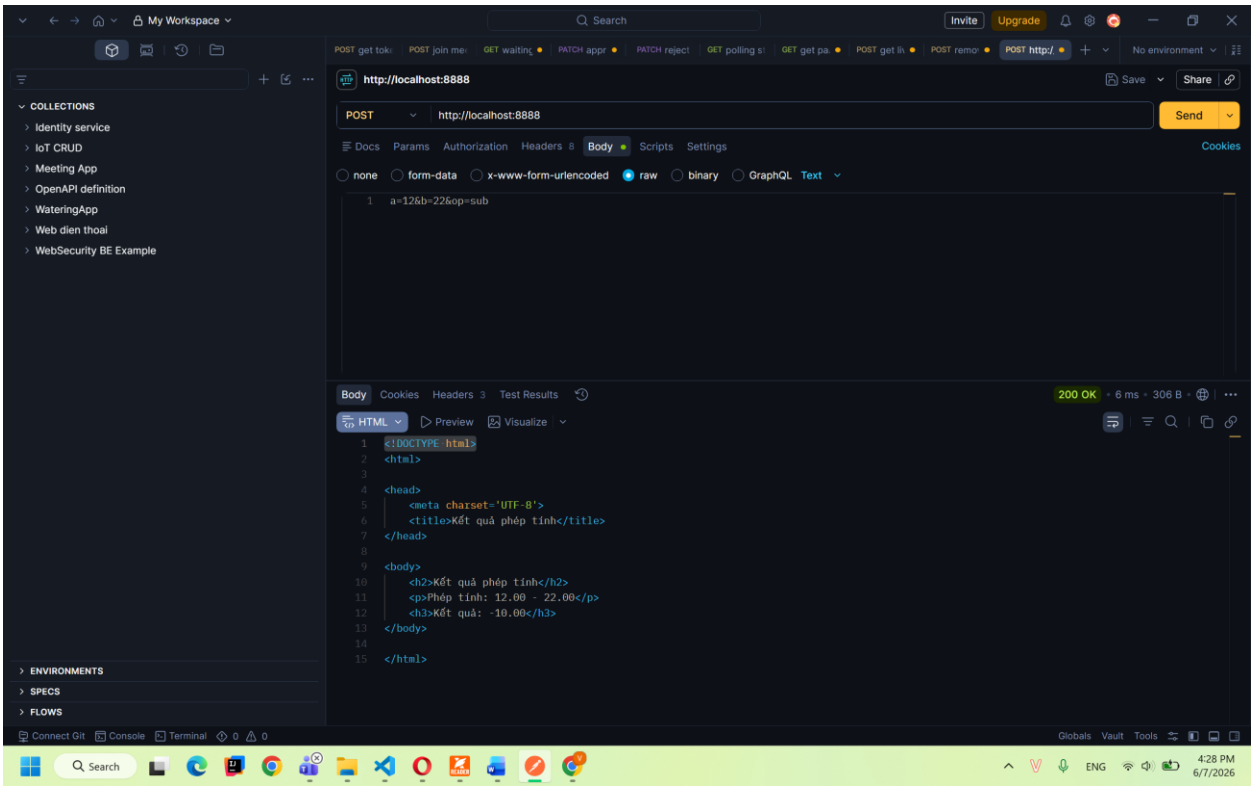
Thiếu tham số. Ví dụ: /?a=10&b=5&op=add



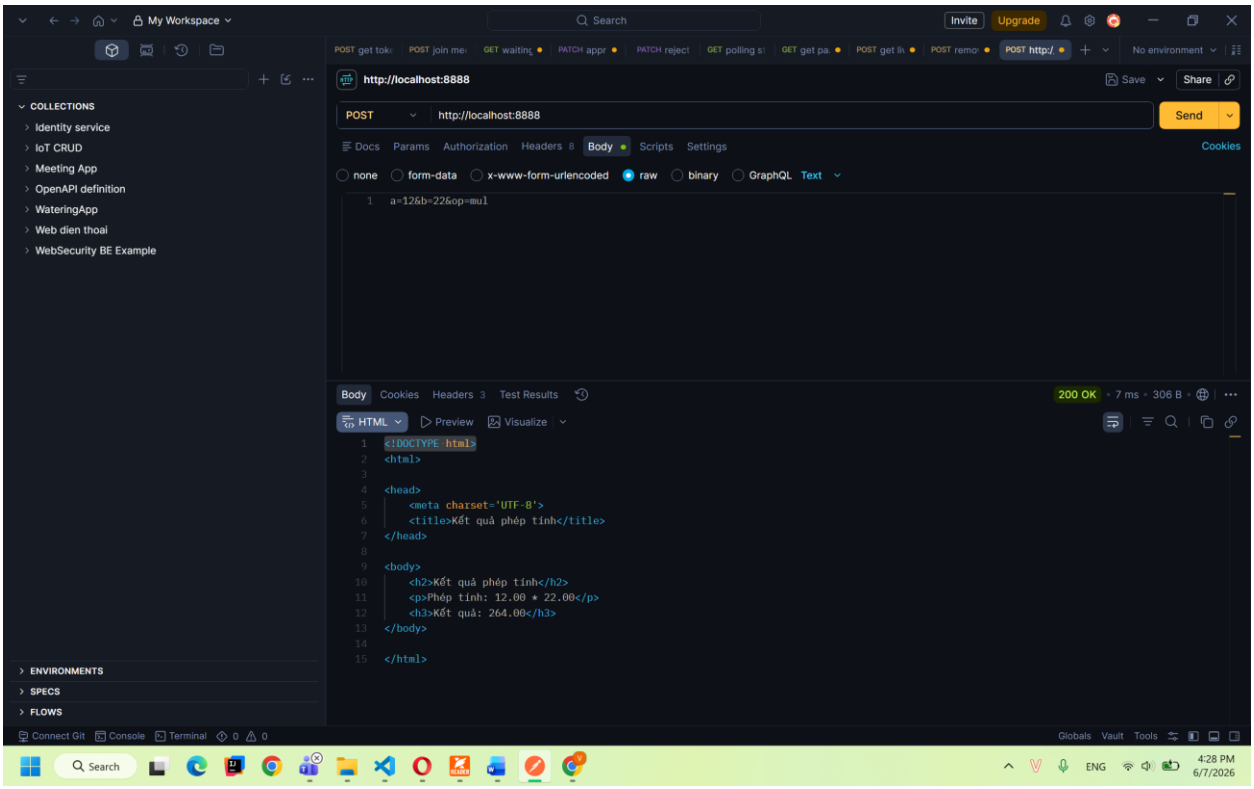
- Tạo post request bằng postman (body: a=12&b=22&op=add)



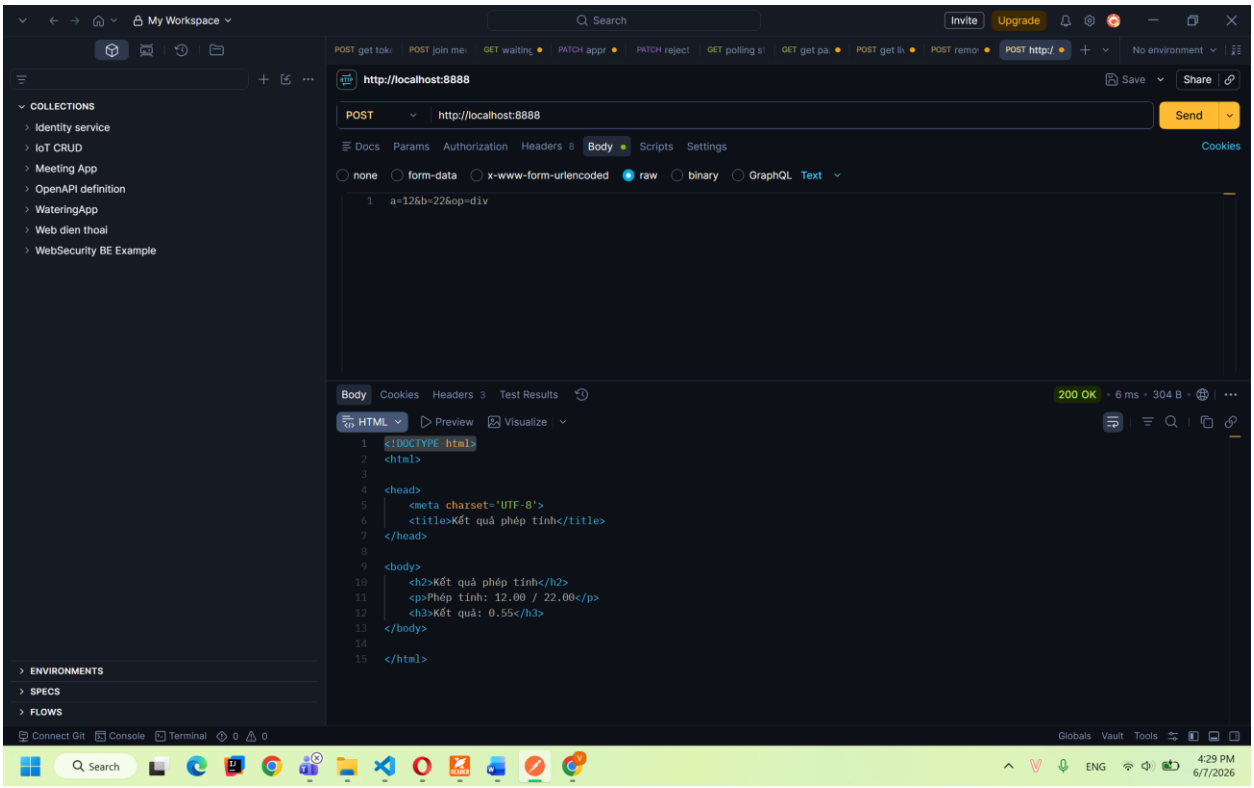
- Tạo post request bằng postman (body: a=12&b=22&op=sub)



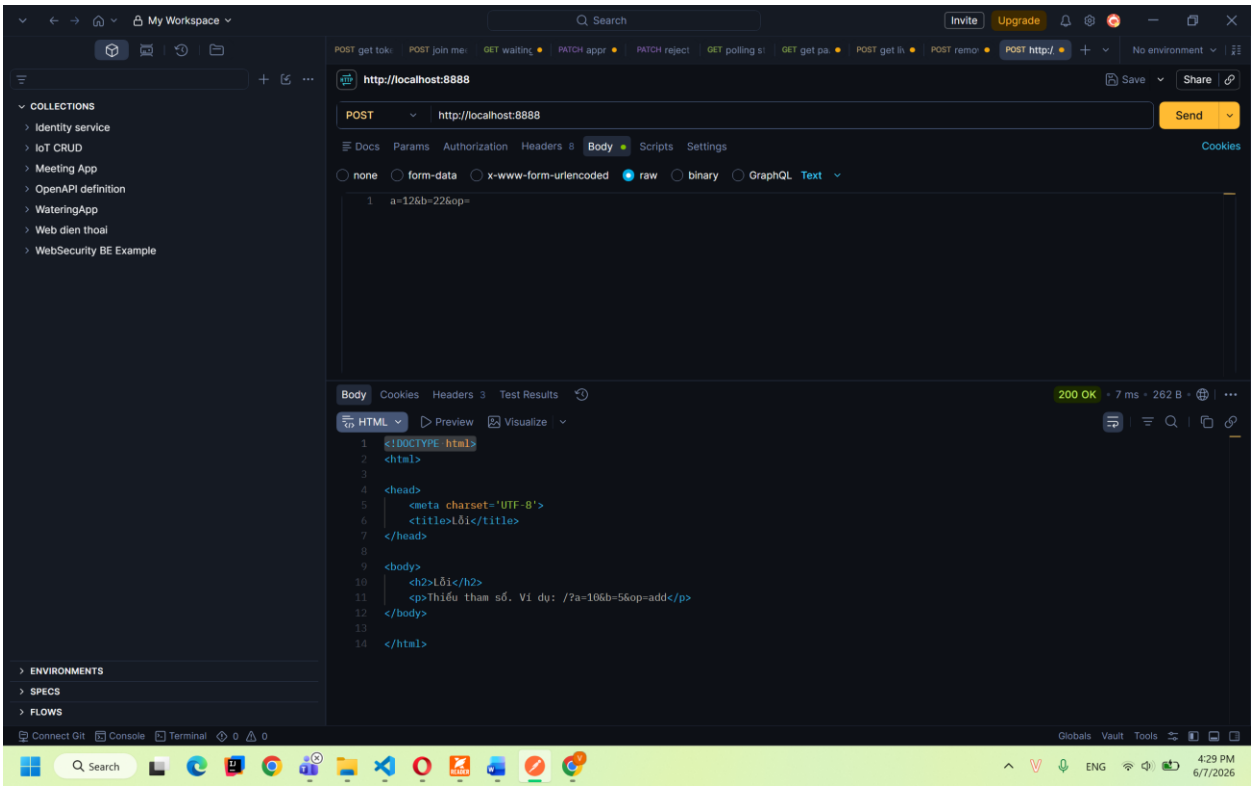
- Tạo post request bằng postman (body: a=12&b=22&op=mul)



- Tạo post request bằng postman (body: a=12&b=22&op=div)



- Tạo post request bằng postman (trường hợp lỗi)



2. Bài 2 – HTTP_file

- Mã nguồn

```c
#include <arpa/inet.h>
#include <ctype.h>
#include <dirent.h>
#include <signal.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

void signalHandler(int signo) { // Xử lý sự kiện tiến trình con kết thúc
    printf("signo = %d\n", signo);

    while (waitpid(-1, NULL, WNOHANG) > 0) {
        printf("A child process terminated.\n");
    }

    return;
}

// ham gui response html ve trinh duyet
void send_response(int client, char *html) {
    char response[8192];

    sprintf(response,
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: text/html; charset=UTF-8\r\n"
            "Content-Length: %ld\r\n"
            "Connection: close\r\n"
            "\r\n"
            "%s",
            strlen(html), html);

    send(client, response, strlen(response), 0);
```

```c
}

// ham gui trang loi ve trinh duyet
void send_error_page(int client, char *message) {
    char html[4096];

    sprintf(html,
            "<!DOCTYPE html>"
            "<html>"
            "<head>"
            "<meta charset='UTF-8'>"
            "<title>Lỗi</title>"
            "</head>"
            "<body>"
            "<h2>Lỗi</h2>"
            "<p>%s</p>"
            "</body>"
            "</html>",
            message);

    send_response(client, html);
}

// ham lay content type dua vao duoi file
char *get_content_type(char *file_path) {
    char *ext = strrchr(file_path, '.');

    if (ext == NULL) {
        return "application/octet-stream";
    }

    // file van ban
    if (strcmp(ext, ".txt") == 0) {
        return "text/plain; charset=UTF-8";
    } else if (strcmp(ext, ".html") == 0 || strcmp(ext, ".htm") == 0) {
        return "text/html; charset=UTF-8";
    } else if (strcmp(ext, ".css") == 0) {
        return "text/css; charset=UTF-8";
    } else if (strcmp(ext, ".js") == 0) {
        return "application/javascript; charset=UTF-8";
    }
```

```
    // file anh
    else if (strcmp(ext, ".jpg") == 0 || strcmp(ext, ".jpeg") == 0) {
        return "image/jpeg";
    } else if (strcmp(ext, ".png") == 0) {
        return "image/png";
    } else if (strcmp(ext, ".gif") == 0) {
        return "image/gif";
    } else if (strcmp(ext, ".webp") == 0) {
        return "image/webp";
    }

    // file audio
    else if (strcmp(ext, ".mp3") == 0) {
        return "audio/mpeg";
    } else if (strcmp(ext, ".wav") == 0) {
        return "audio/wav";
    } else if (strcmp(ext, ".ogg") == 0) {
        return "audio/ogg";
    }

    // file video
    else if (strcmp(ext, ".mp4") == 0) {
        return "video/mp4";
    } else if (strcmp(ext, ".webm") == 0) {
        return "video/webm";
    } else if (strcmp(ext, ".avi") == 0) {
        return "video/x-msvideo";
    }

    return "application/octet-stream";
}

// ham gui file ve trinh duyet
void send_file(int client, char *file_path) {
    FILE *file = fopen(file_path, "rb");

    if (file == NULL) {
        send_error_page(client, "Không thể mở file.");
        return;
    }
```

```
    // lay kich thuoc file
    fseek(file, 0, SEEK_END);
    long file_size = ftell(file);
    rewind(file);

    char header[1024];

    sprintf(header,
            "HTTP/1.1 200 OK\r\n"
            "Content-Type: %s\r\n"
            "Content-Length: %ld\r\n"
            "Connection: close\r\n"
            "\r\n",
            get_content_type(file_path), file_size);

    send(client, header, strlen(header), 0);

    // doc file va gui tung phan ve trinh duyet
    char buffer[4096];
    int bytes_read;

    while ((bytes_read = fread(buffer, 1, sizeof(buffer), file)) > 0) {
        send(client, buffer, bytes_read, 0);
    }

    fclose(file);
}

// ham doi ky tu dac biet trong URL
void url_decode(char *src, char *dest) {
    char a, b;

    while (*src) {
        if ((*src == '%') && ((a = src[1]) && (b = src[2])) && isxdigit(a) &&
            isxdigit(b)) {
            if (a >= 'a') {
                a -= 'a' - 'A';
            }
            if (a >= 'A') {
                a = a - 'A' + 10;
```

```
            } else {
                a -= '0';
            }

            if (b >= 'a') {
                b -= 'a' - 'A';
            }
            if (b >= 'A') {
                b = b - 'A' + 10;
            } else {
                b -= '0';
            }

            *dest++ = 16 * a + b;
            src += 3;
        } else if (*src == '+') {
            *dest++ = ' ';
            src++;
        } else {
            *dest++ = *src++;
        }
    }

    *dest = '\0';
}

// ham gui danh sach thu muc va file ve trinh duyet
void send_directory_page(int client, char *dir_path, char *url_path) {
    DIR *dir = opendir(dir_path);

    if (dir == NULL) {
        send_error_page(client, "Không thể mở thư mục.");
        return;
    }

    char html[65536];

    sprintf(html,
            "<!DOCTYPE html>"
            "<html>"
            "<head>"
```

```
            "<meta charset='UTF-8'>"
            "<title>HTTP File Server</title>"
            "</head>"
            "<body>"
            "<h2>Danh sách thư mục: %s</h2>"
            "<ul>",
            url_path);

    // neu khong phai thu muc goc thi them link quay lai
    if (strcmp(url_path, "/") != 0) {
        strcat(html, "<li><a href='../'><b>..</b></a></li>");
    }

    struct dirent *entry;

    while ((entry = readdir(dir)) != NULL) {
        // bo qua thu muc hien tai va thu muc cha
        if (strcmp(entry->d_name, ".") == 0 ||
            strcmp(entry->d_name, "..") == 0) {
            continue;
        }

        char item_path[2048];
        sprintf(item_path, "%s/%s", dir_path, entry->d_name);

        struct stat st;

        if (stat(item_path, &st) < 0) {
            continue;
        }

        char href[2048];

        if (strcmp(url_path, "/") == 0) {
            sprintf(href, "/%s", entry->d_name);
        } else {
            sprintf(href, "%s/%s", url_path, entry->d_name);
        }

        // neu la thu muc thi in dam
        if (S_ISDIR(st.st_mode)) {
```

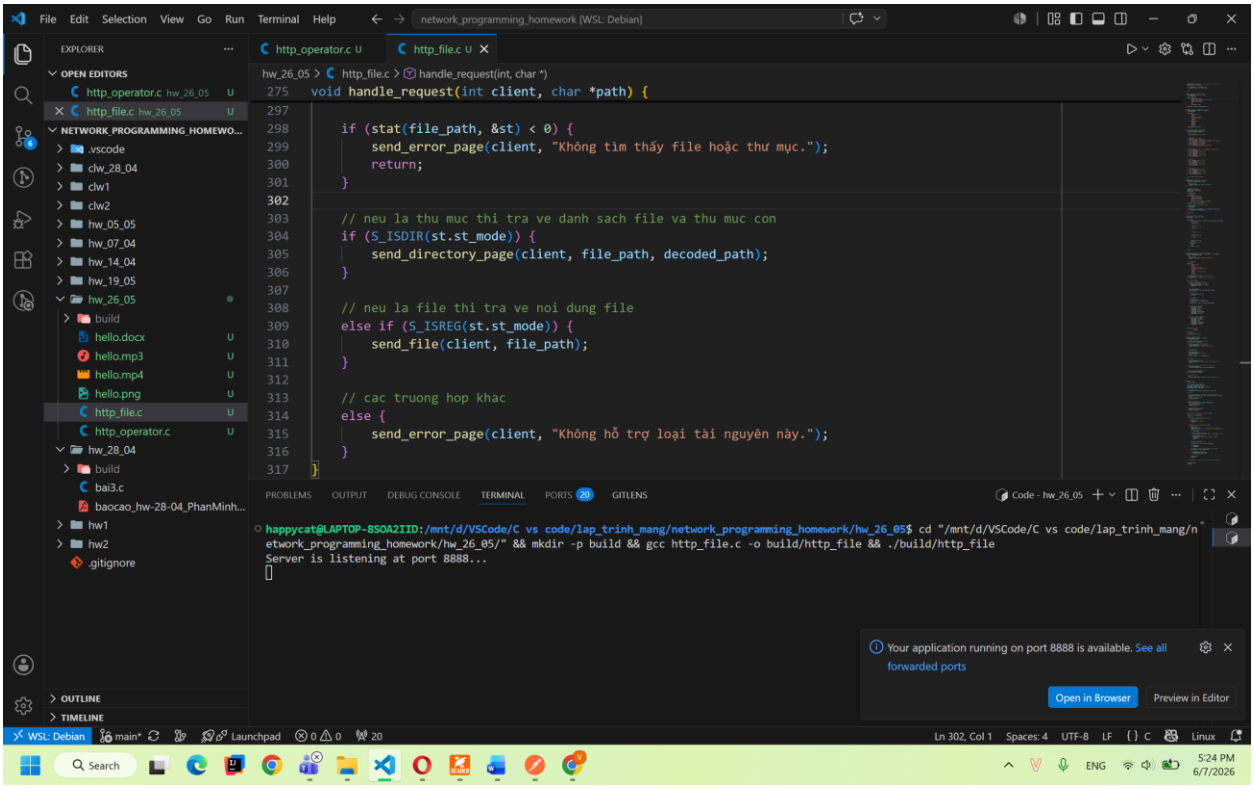
```
            strcat(html, "<li>");
            strcat(html, "<a href='");
            strcat(html, href);
            strcat(html, "/'><b>");
            strcat(html, entry->d_name);
            strcat(html, "/</b></a>");
            strcat(html, "</li>");
        }

        // neu la file thi in nghieng
        else if (S_ISREG(st.st_mode)) {
            strcat(html, "<li>");
            strcat(html, "<a href='");
            strcat(html, href);
            strcat(html, "'><i>");
            strcat(html, entry->d_name);
            strcat(html, "</i></a>");
            strcat(html, "</li>");
        }
    }

    strcat(html, "</ul>"
                 "</body>"
                 "</html>");

    closedir(dir);

    send_response(client, html);
}

// ham xu ly request tu client
void handle_request(int client, char *path) {
    char decoded_path[2048];

    // doi url encoding thanh ky tu binh thuong
    url_decode(path, decoded_path);

    // chan truy cap ra ngoai thu muc hien tai
    if (strstr(decoded_path, "..") != NULL) {
        send_error_page(client, "Đường dẫn không hợp lệ.");
        return;
```

```
    }

    char file_path[2048];

    // neu request thu muc goc
    if (strcmp(decoded_path, "/") == 0) {
        strcpy(file_path, ".");
    } else {
        sprintf(file_path, ".%s", decoded_path);
    }

    struct stat st;

    if (stat(file_path, &st) < 0) {
        send_error_page(client, "Không tìm thấy file hoặc thư mục.");
        return;
    }

    // neu la thu muc thi tra ve danh sach file va thu muc con
    if (S_ISDIR(st.st_mode)) {
        send_directory_page(client, file_path, decoded_path);
    }

    // neu la file thi tra ve noi dung file
    else if (S_ISREG(st.st_mode)) {
        send_file(client, file_path);
    }

    // cac truong hop khac
    else {
        send_error_page(client, "Không hỗ trợ loại tài nguyên này.");
    }
}

int main() {
    int port = 8888;

    // khai bao dia chi tai cong 8888
    struct sockaddr_in server_addr = {0};
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

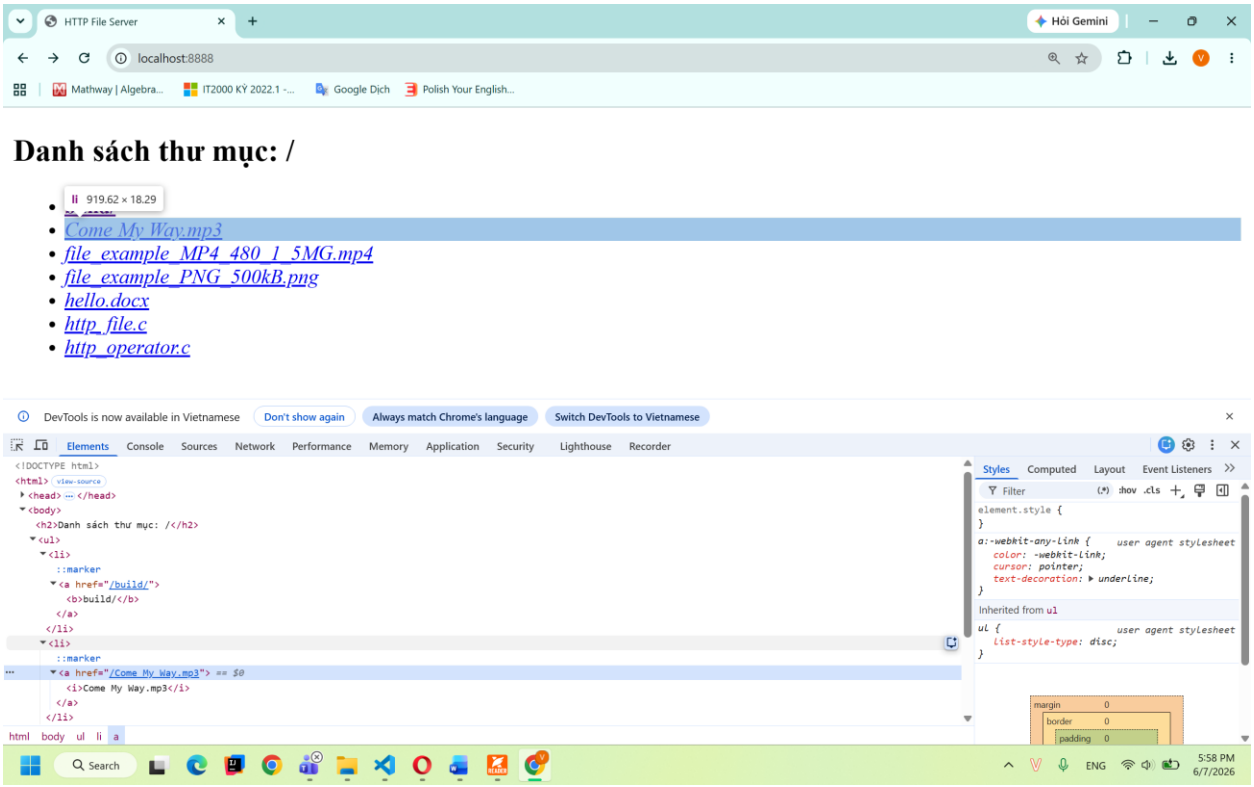
```c
    server_addr.sin_port = htons(port);

    // tao listen sock
    int listen_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    int opt = 1;

    // Thiết lập SO_REUSEADDR để giải phóng cổng ngay khi server tắt
    if (setsockopt(listen_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) <
        0) {
        perror("setsockopt failed");
        exit(EXIT_FAILURE);
    }

    // gan dia chi cho socket
    if (bind(listen_sock, (struct sockaddr *)&server_addr,
             sizeof(server_addr)) < 0) {
        perror("bind() failed");
        exit(EXIT_FAILURE);
    }

    if (listen(listen_sock, 5) < 0) {
        perror("listen() failed");
        exit(EXIT_FAILURE);
    }

    printf("Server is listening at port %d...\n", port);

    signal(SIGCHLD, signalHandler);

    while (1) {
        int client = accept(listen_sock, NULL, NULL);

        if (client < 0) {
            perror("accept() failed");
            continue;
        }

        printf("New client accepted: %d\n", client);

        if (fork() == 0) {
```

```
            close(listen_sock);       // dong listen_sock vi khong dung
            signal(SIGCHLD, SIG_DFL); // tien trinh con khong quan tam SIGCHLD

            char buffer[4096];

            // nhan HTTP request tu client
            int bytes_received = recv(client, buffer, sizeof(buffer) - 1, 0);

            if (bytes_received <= 0) {
                printf("Client number %d disconnected\n", client);
                close(client);
                exit(0);
            }

            buffer[bytes_received] = '\0';

            printf("Request from client:\n%s\n", buffer);

            char method[16];
            char path[1024];

            // lay method va path tu dong dau tien cua HTTP request
            sscanf(buffer, "%s %s", method, path);

            // file server chi xu ly GET
            if (strcmp(method, "GET") == 0) {
                handle_request(client, path);
            }

            close(client);
            exit(0);
        }

        close(client); // dong client vi tien trinh cha khong dung den
    }

    close(listen_sock);
    return 0;
}
```

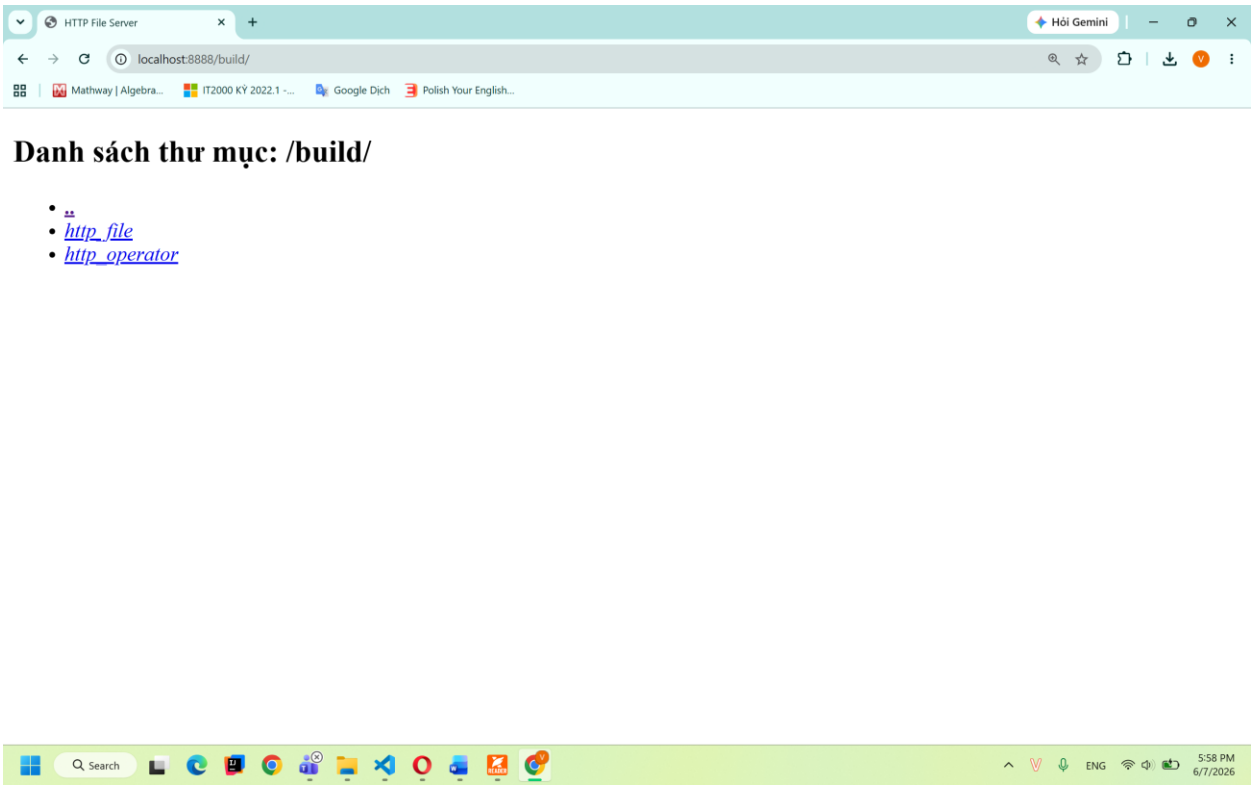
- Kết quả
    - Chạy server ở cổng 8888



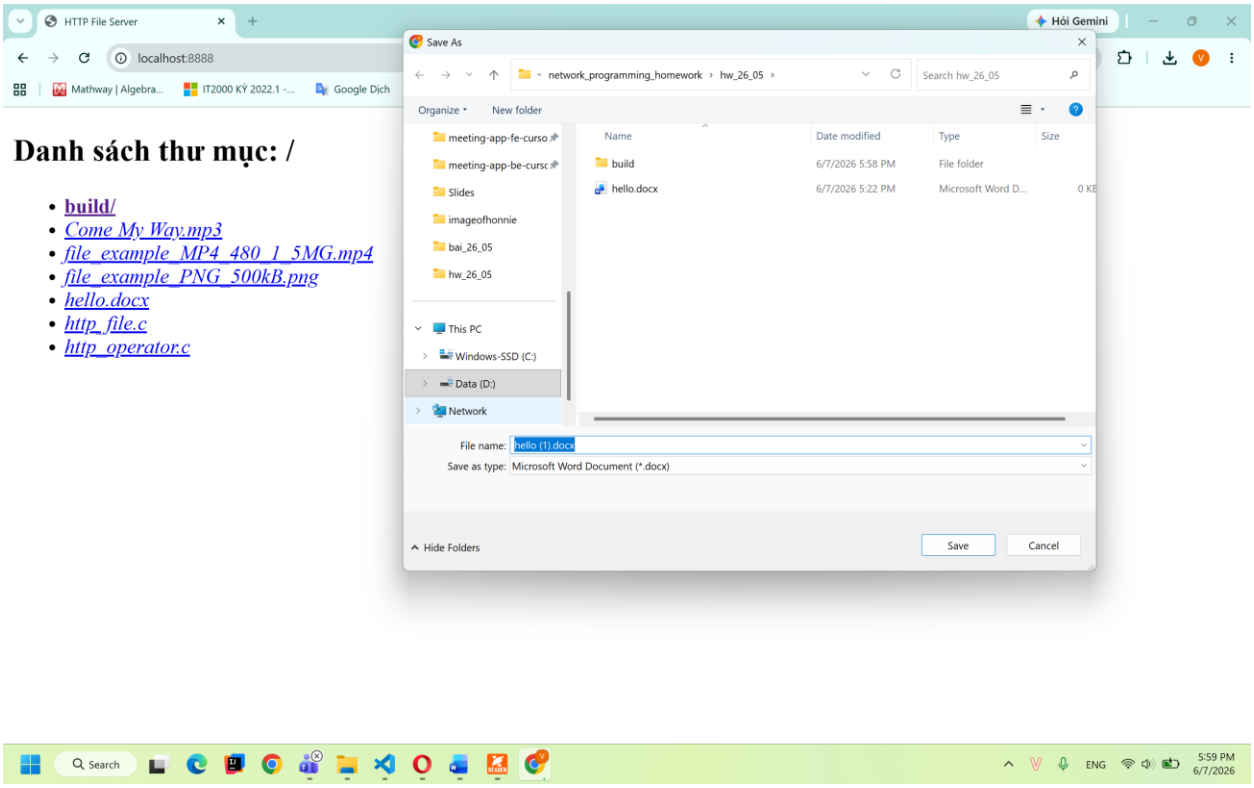
    - Mở trình duyệt và truy cập localhost:8888 sẽ nhận được trang html chứa danh sách thư mục và tệp, thư mục in đậm còn tệp in nghiêng



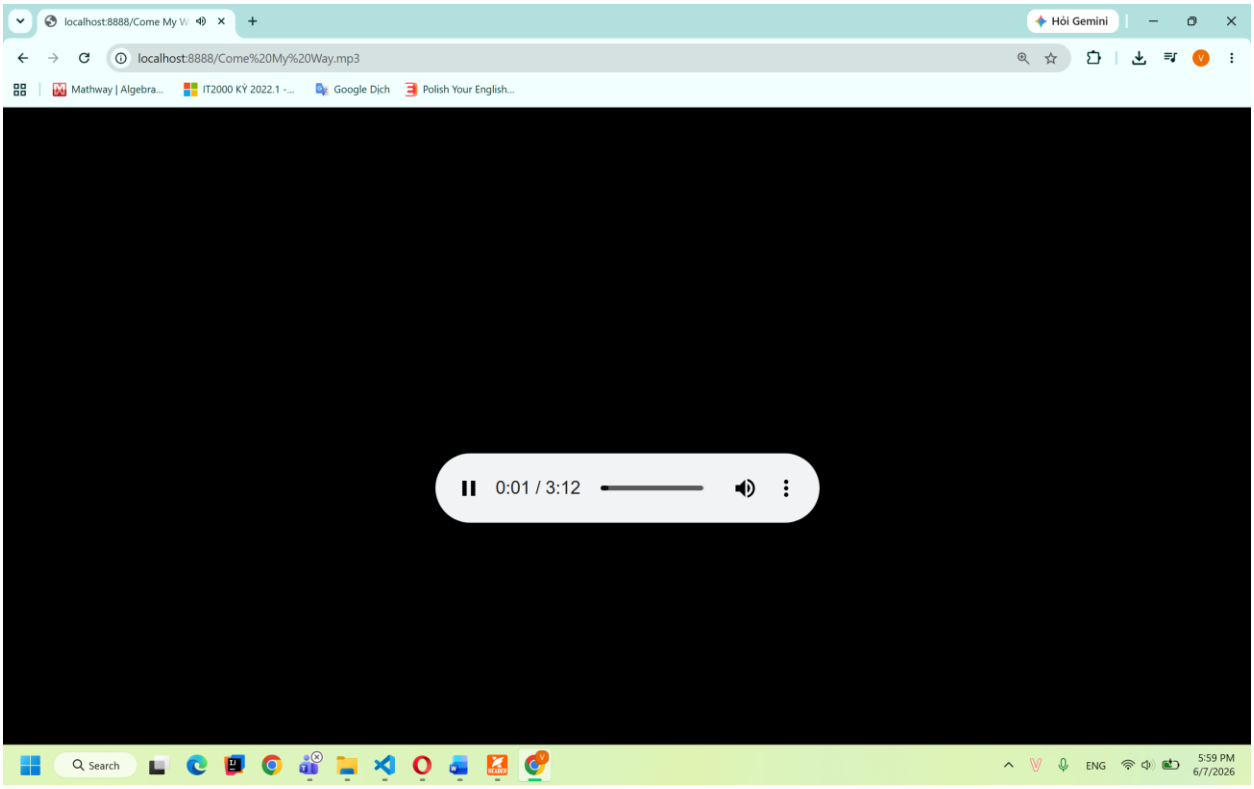
- Nhấn vào tên thư mục sẽ trả về danh sách các file và thư mục con, nhấn vào .. để quay lại thư mục gốc



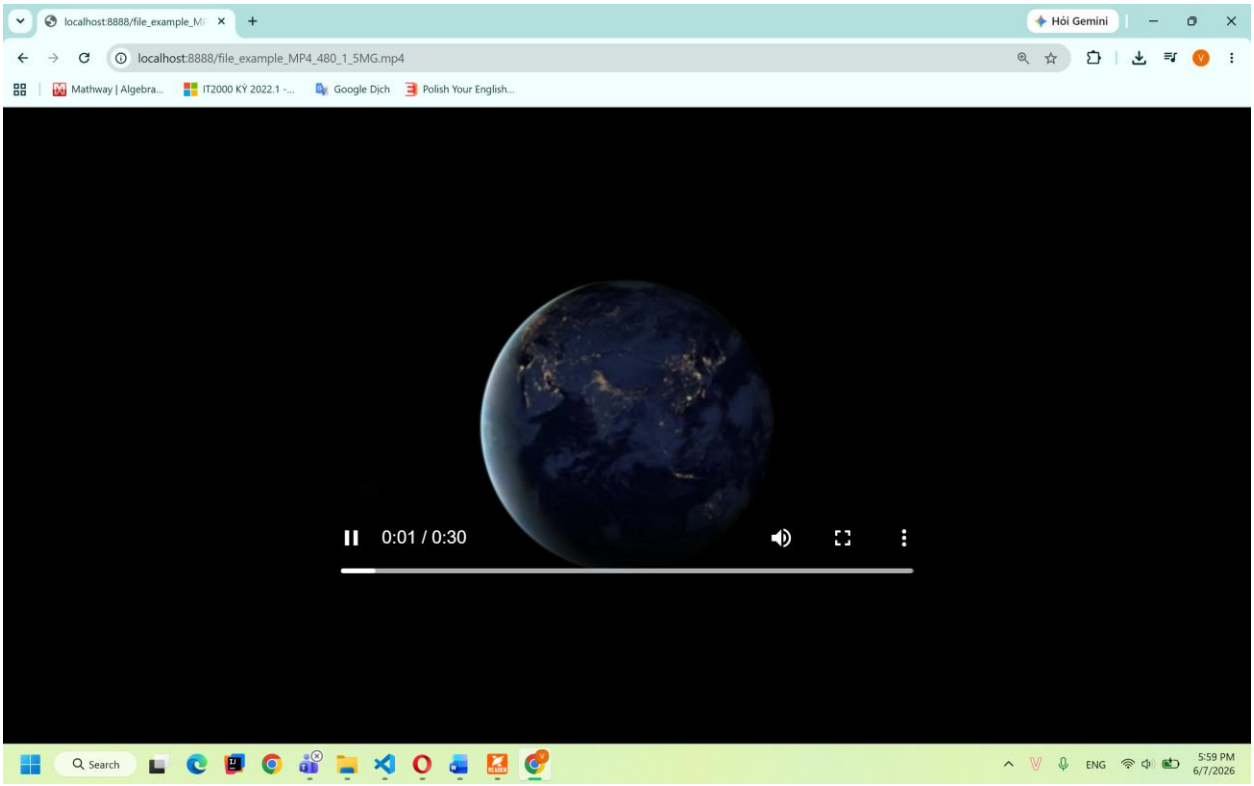
- Nhấn vào file sẽ nhận được nội dung file (dạng văn bản)



- Nhấn vào file sẽ nhận được nội dung file (dạng audio)



- Nhấn vào file sẽ nhận được nội dung file (dạng video)



- Nhấn vào file sẽ nhận được nội dung file (dạng ảnh)